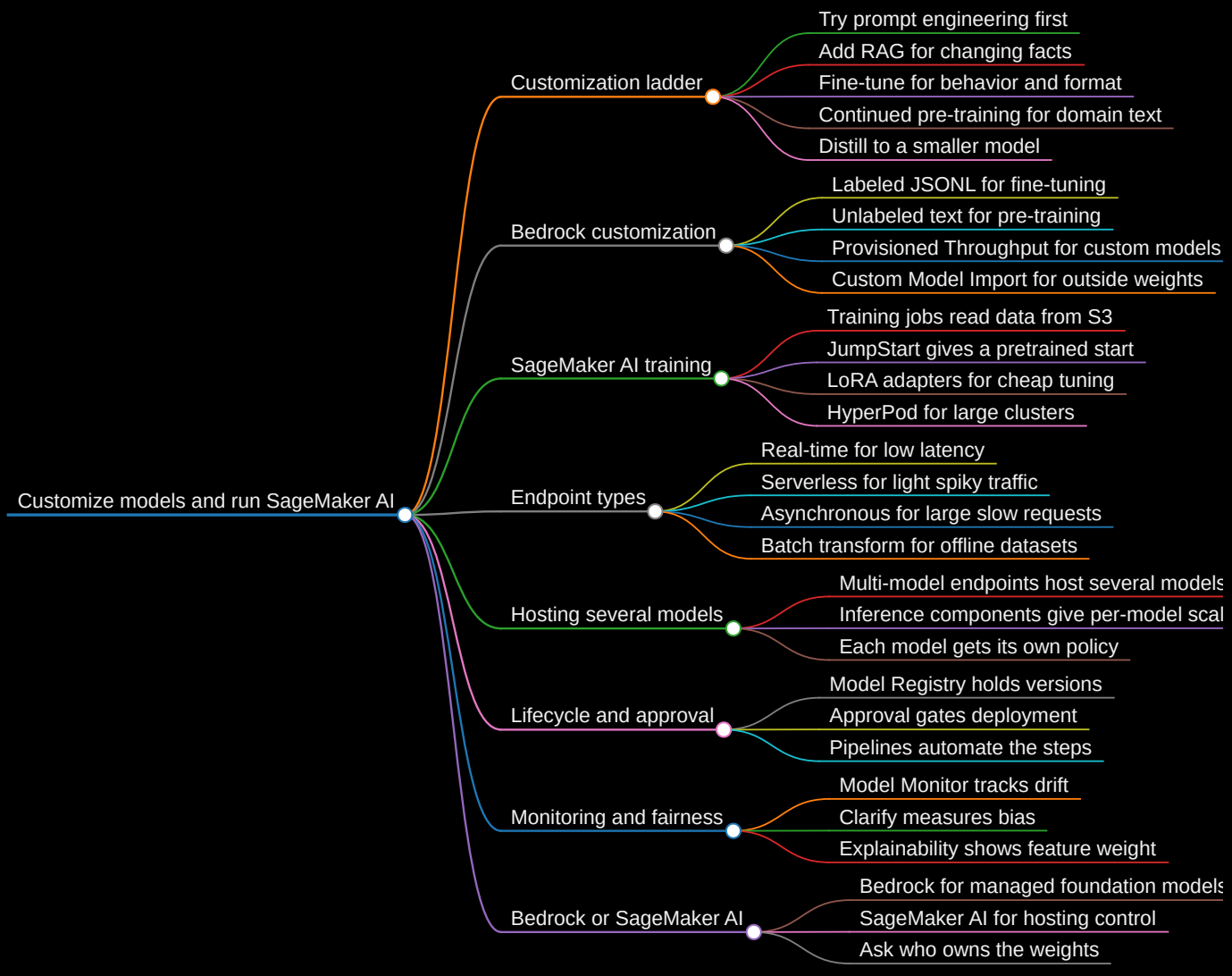
Customize models and run SageMaker AI

After this module you can pick the right customization method for a scenario, the right SageMaker AI endpoint type for a traffic pattern, and the right lifecycle feature for a release process. You will find the deciding constraint first. Then you will reject each wrong option with one reason. Most long scenarios hide the answer in one sentence about what must change or how requests arrive.



The deciding constraint

Start from the scenario, not from the options. Long questions give one constraint that decides the answer. Other details describe the company, the team, or the history, and you can set them aside. Write the constraint in three or four words before you read any option.

Customization questions turn on what must change. Facts that change often point to retrieval. Behavior, tone, or output format points to tuning. Domain text without labels points to continued pre-training. A smaller model that must copy a larger one points to distillation.

Hosting questions turn on the request pattern. Payload size, response time, traffic shape, and offline work each point to a different endpoint type. A scenario that mentions a large file and a long wait points away from real-time hosting.

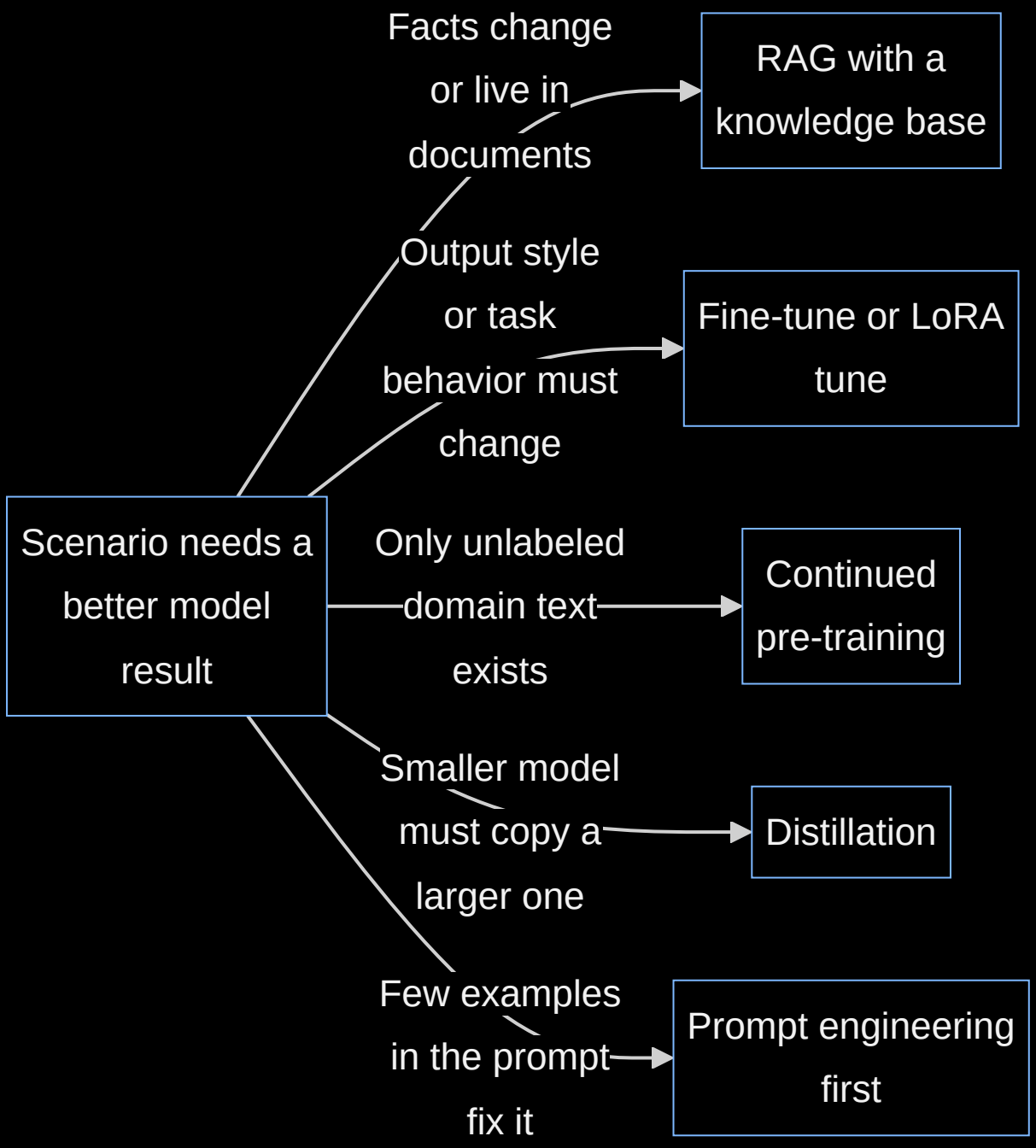


Figure: The first split is what must change, the facts or the model itself.

Signal words for customization

- **Signal:** "latest policy", "weekly catalog", "private documents", "must cite the source". Rule: Use retrieval when knowledge changes often or must be cited.
- **Signal:** "always reply in this format", "adopt the house tone", "labeled examples". Rule: Use fine-tuning when behavior or format must change.
- **Signal:** "large archive of unlabeled text", "no task labels", "domain vocabulary". Rule: Use continued pre-training when text is plentiful and labels are absent.

- **Signal:** "cheaper model", "same answers on one task", "reduce cost per call". Rule: Use distillation when a small model must imitate a larger teacher.
- **Signal:** "quick test", "a few examples", "no training budget". Rule: Use prompt engineering first because it costs the least to test.

The ladder runs from cheap to expensive. Each step adds data work, training cost, and operating burden. A team that skips to fine-tuning often pays for a problem that a better prompt or a retrieval step would have solved. The exam reads this order as a cost signal. Choose the lowest step that meets the stated requirement.

Exam tip: Look for the word that describes change. Words such as "latest", "updated", or "weekly" describe facts. Words such as "tone", "format", or "style" describe behavior. The two words lead to different services.

Customization methods in scope

Prompt engineering

Fact: Prompt engineering changes the instructions and examples sent with each request. The model weights do not change. It costs nothing to train and is the first step to test.

Rule: Test prompt engineering first when a few examples or clear instructions fix the output.

Trap: A long prompt does not add knowledge that the model never learned. It also raises token cost on every call, which fine-tuning can reduce over time.

Prompt engineering pairs with retrieval and with guardrails. Retrieval supplies the facts. Guardrails enforce the limits. Limits and token caps depend on the model, and the fetched sources do not list them. Not in source.

Source: Sundog course slide 25, https://docs.aws.amazon.com/bedrock/latest/userguide/custom-models.html

Retrieval Augmented Generation

Fact: Retrieval finds relevant text at request time and adds it to the prompt. The model weights stay the same. Bedrock Knowledge Bases and OpenSearch Serverless can back this pattern. Semantic search uses vectors. Hybrid search also uses keyword fields.

Rule: Use RAG when facts change weekly or when the answer must point to a source document.

Trap: Retrieval does not change tone, format, or task skill. A RAG design that fails on output format needs tuning, not a bigger index.

Hybrid search needs filterable keyword fields in the index, according to the course slides. Chunking strategy strongly affects relevance. The course notes also say that a serverless OpenSearch collection does not scale to zero, so it bills while idle. Index size limits are Not in source.

Source: Sundog course slide 139, https://docs.aws.amazon.com/bedrock/latest/userguide/custom-models.html

Fine-tuning in Bedrock

Fact: A Bedrock customization job uses labeled training pairs. Supervised fine-tuning adjusts model parameters so outputs match the labeled examples. Training files sit in Amazon S3. Bedrock charges for training by tokens processed and for monthly model storage.

For non-conversational tasks, each JSON line holds a prompt field and a completion field. For conversational models that use the Converse message format, each line holds system text and an array of user and assistant messages. The file type is JSONL, so one line is one record. Use about six characters per token when you estimate dataset size.

Fact: Limits vary by base model. Titan Text G1 Express allows 4,096 tokens per sample at batch size 1, a 1 GB training file, and a 100 MB validation file. Llama 3.1 needs at least 100 records, allows up to 10,000 records, and caps input plus output at 16,000 tokens.

Fact: For Claude 3 Haiku, the minimum is 32 records, the training file can reach 10 GB, and the validation file can reach 1 GB. Image generation and image embedding models do not support a validation set.

Rule: Choose fine-tuning when the model must learn a task pattern from labeled examples and no fact lookup can solve it.

Trap: Fine-tuning does not add facts that change each week. Those facts go stale inside the weights. Pair the tuned model with retrieval instead.

The course slides say that training data for sensitive content can use a VPC and PrivateLink. The slides also warn that fine-tuning can get expensive. A fine-tuned model is used like any other model after training, but it has its own pricing and capacity rules. Those rules appear in the Provisioned Throughput section below.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/custom-models.html and https://docs.aws.amazon.com/bedrock/latest/userguide/model-customization-prepare.html and Sundog course slide 26

Continued pre-training

Fact: Continued pre-training feeds unlabeled text to a base model. The model learns the vocabulary and style of the domain. The course slides describe it as fine-tuning with unlabeled data, where each record has a single input field. The records do not need a task label.

Rule: Choose continued pre-training when a large archive of domain text exists and no task labels exist.

Trap: Continued pre-training does not teach a required answer format. A model trained only on raw text still needs fine-tuning or prompts to answer in the right shape.

Continued pre-training is a broader change than fine-tuning. It touches much more of the model for each record. Teams use it for language gaps, not for a

single classification task. Minimum records, token caps, and file sizes for this method are Not in source in the fetched pages.

Source: Sundog course slide 27, https://docs.aws.amazon.com/bedrock/latest/userguide/custom-models.html

Reinforcement fine-tuning and preference tuning

Fact: Reinforcement learning from human feedback trains a model with a reward signal rather than a fixed answer. Bedrock offers reinforcement fine-tuning. You provide prompts or existing invocation logs, and you define reward functions in AWS Lambda that score each response.

Rule: Choose a reward-based method when quality is measured by a score rather than one correct label.

Trap: A reward method is not the first step. Teams that skip prompt tests and retrieval spend more on training to reach the same result.

Preference tuning uses pairs of answers, where one answer is marked better. Its exact Bedrock name and limits are Not in source. The reward pattern is the part the fetched docs describe. Treat preference data as a labeled signal and reward functions as a scored signal.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/custom-models.html

Distillation

Fact: Distillation moves behavior from a teacher model to a smaller student model. In Bedrock you pick both models, then supply prompts. Bedrock generates teacher answers and fine-tunes the student. You can instead supply labeled prompt and answer pairs, or reuse teacher answers from invocation logs.

Fact: The distillation dataset can grow up to 15,000 prompt and response pairs when Bedrock uses its data synthesis. Teacher calls for synthesis bill at on-demand rates. Only the training split fine-tunes the student.

Rule: Choose distillation when a smaller, cheaper model must match a larger model on one narrow task.

Trap: When you reuse teacher answers from invocation logs as pairs, the teacher in the job must match the model in the logs. A mismatch means the logs are not used.

Distillation differs from compression. Quantization and pruning shrink a model. Knowledge distillation trains a smaller model from a larger one. The course slides list all three as ways to optimize a model, and the choice depends on the quality target.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/model-distillation.html and Sundog course slide 319

Bedrock Custom Model Import

Fact: Custom Model Import brings a model you customized outside Bedrock into Bedrock. You supply Hugging Face format weights in safetensors form, with config and tokenizer files, in S3. Bedrock detects the architecture during the import job. You invoke the result on demand with InvokeModel or InvokeModelWithResponseStream.

Fact: Supported families include Mistral, Mixtral, Flan, Llama 2 and Llama 3 series, GPTBigCode, Qwen2 and Qwen2.5, and GPT-OSS. Text model weights must be under 200 GB. Multimodal weights must be under 100 GB. Context length must be under 128K. Embedding models are not supported.

Fact: At the time of the fetch, the page listed four regions: eu-central-1, us-east-1, us-east-2, and us-west-2. Batch inference and CloudFormation do not work with imported models.

Rule: Choose Custom Model Import when you own the weights, the architecture is supported, and the app calls Bedrock.

Trap: Import does not train anything. It hosts weights that you already trained, so a request to tune a model in Bedrock needs a customization job instead.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/model-customization-import-model.html and Sundog course slide 317

SageMaker AI training jobs

Fact: A SageMaker AI training job runs your code or a built-in algorithm in a container on managed instances. Input data comes from S3. The trained model artifact goes back to S3. The container image lives in Amazon ECR. Processing jobs prepare data with the same pattern: input from S3, output to S3.

Rule: Choose SageMaker AI training when you need custom code, a custom container, or control of the training loop.

Trap: A training job does not create an endpoint. Deploy the artifact as a separate step, then choose the endpoint type.

Data Wrangler gives a visual preparation flow in SageMaker Studio with more than 300 built-in transforms. It also accepts custom transforms in pandas or PySpark. The course slides say the endpoint must format its own input. SageMaker AI will not reshape a JSON request for you. Job limits are Not in source.

Source: Sundog course slides 78, 92, 308, 311, 312

LoRA and parameter-efficient tuning

Fact: LoRA adds small low-rank matrices to the model, usually beside the attention weights. Training updates only those small matrices. The base model stays unchanged. At inference, the small matrices are added to the base weights. Storage, training, and inference costs fall as a result.

Fact: LoRA is different from an adapter layer. An adapter layer inserts new layers into the model. LoRA adds weight matrices that merge with the existing weights. PEFT is the umbrella term for methods like this, and the fetched sources do not define it further. Not in source.

Rule: Choose LoRA when many tuned versions of one large base model must be stored and deployed cheaply.

Trap: A LoRA adapter is not a standalone model. It always needs its base model at inference, so the registry must link adapter and base.

Full copies of large models run to many gigabytes, and the size depends on numeric precision. A small adapter file is far smaller. Versioning adapters over one base saves storage and makes rollback a matter of pointing at a different adapter. The exact size of an adapter depends on rank and layers. Not in source.

Source: Sundog course slide 28

SageMaker JumpStart

Fact: JumpStart gives pretrained open models, solution templates, and example notebooks. You can fine-tune, deploy, and evaluate a model from SageMaker Studio or from the SDK. Models come from public and private hubs. The course slide cites over 150 open source models.

Rule: Choose JumpStart when you want a pretrained open model with a managed starting point and a deploy button.

Trap: JumpStart is a catalog and a workflow. Its deployments still run on SageMaker AI endpoints, so the endpoint type rules still apply.

The page warns that you must review license terms before you use third-party content. On March 13, 2026, AWS delisted some models from the catalog. Existing endpoints for those models still work. A scenario that depends on a delisted model needs a different starting point.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/studio-jumpstart.html

SageMaker HyperPod

Fact: HyperPod provisions resilient clusters of accelerators for large training and inference work. It monitors cluster instances and replaces faulty hardware.

Orchestration options include Slurm and Amazon EKS. The service targets foundation model training across thousands of accelerators.

Rule: Choose HyperPod when many accelerators must run one large training job for a long time and failures must be handled for you.

Trap: One fine-tuning job on a few GPUs does not need HyperPod. A standard training job is simpler and cheaper for that scale.

HyperPod appears in the fetched docs with a region list that is longer than the list for Custom Model Import. Instance counts and per-cluster limits are Not in source. The course slides mention UltraServers, which connect instances with a fast accelerator link. That is the hardware side of the same large-scale story.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/sagemaker-hyperpod.html and Sundog course slide 301

Endpoint types for SageMaker AI hosting

SageMaker AI offers several ways to host a model. Each endpoint type sets a payload limit, a time limit, and a billing shape. Read the limits before you read the options. Most wrong answers pick a type whose limit the scenario breaks.

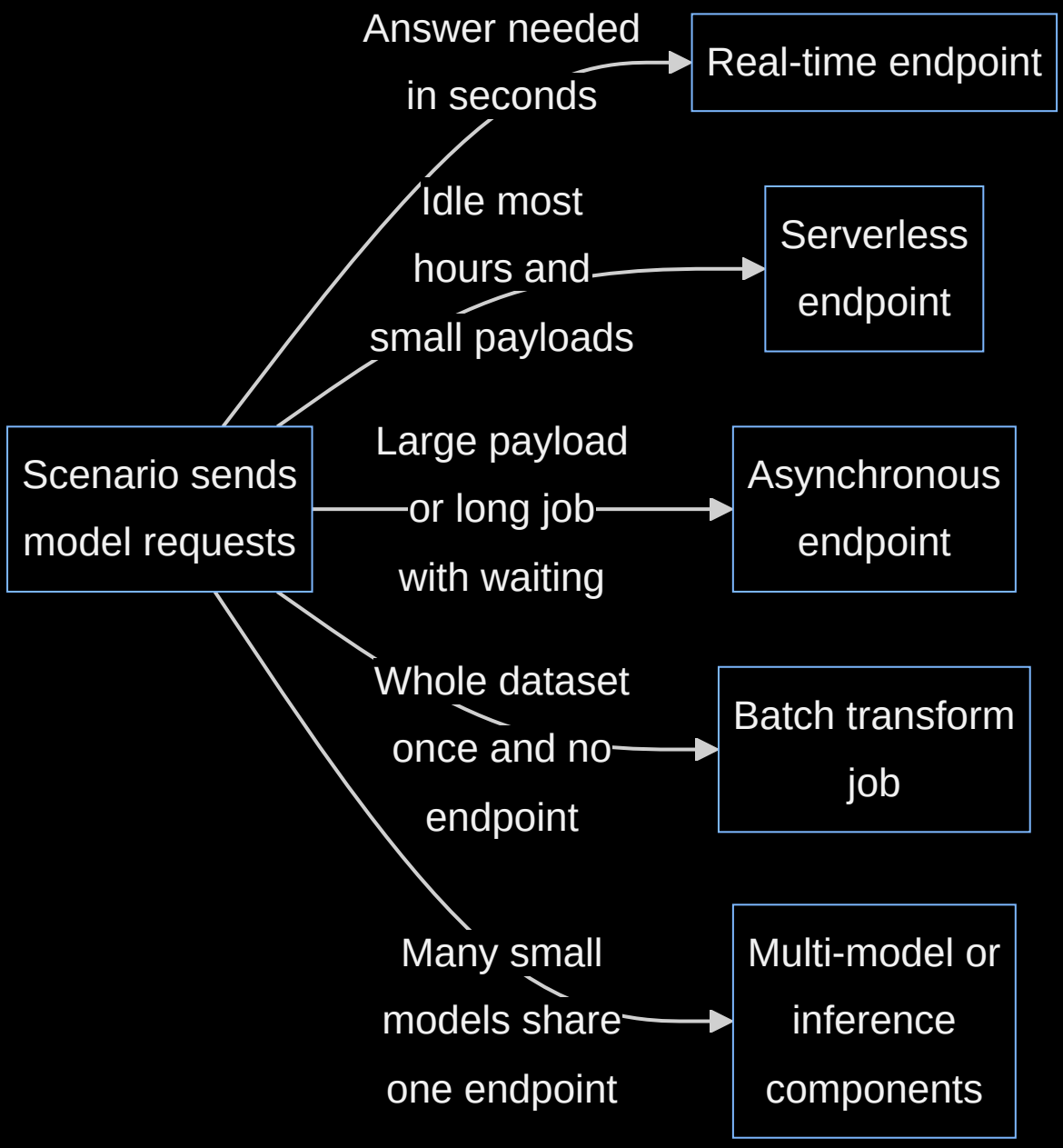


Figure: The request pattern decides the endpoint type, not the model size alone.

Real-time endpoints

Fact: A real-time endpoint is a persistent, fully managed REST endpoint. It supports autoscaling and production variants. The payload limit is 25 MB. Regular responses must finish within 60 seconds. Streaming responses can run up to 8 minutes.

Fact: The course slides say SageMaker AI can deploy models up to 500 GB. Raise the container health check and the download timeout so the container can load large weights. Large models often use GPU instances such as ml.p4d.24xlarge. Small named entity models can use CPU instances such as ml.c5.9xlarge.

Rule: Choose real-time when the caller waits for each answer and each payload stays under 25 MB.

Trap: A bigger instance does not raise the 60-second response limit. A job that runs for several minutes needs an asynchronous or batch design.

Deployment guardrails apply to real-time and asynchronous endpoints. They include all-at-once, canary, and linear traffic shifts, with auto rollback. Shadow tests send a copy of traffic to a new variant and compare results without answering users. The course slides also note that you must watch the metrics yourself before you promote.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/deploy-model-options.html and https://docs.aws.amazon.com/sagemaker/latest/dg/realtime-endpoints.html and Sundog course slides 300, 313, 314

Serverless Inference

Fact: A serverless endpoint scales to zero and bills per use. The payload limit is 4 MB. Processing can run up to 60 seconds. Memory options are 1024, 2048, 3072, 4096, 5120, and 6144 MB. Each endpoint has 5 GB of ephemeral disk.

Fact: Serverless does not support GPUs, VPC configuration, network isolation, data capture, multiple production variants, Model Monitor, inference pipelines, or multi-model endpoints. Per endpoint maximum concurrency is 200. Regional concurrency is 500 or 1000 depending on the region. Container images can reach 10 GB. Provisioned concurrency keeps the endpoint warm.

Rule: Choose serverless for light or unpredictable traffic that accepts cold starts and payloads under 4 MB.

Trap: Serverless has no GPU. A scenario with a GPU model and a serverless answer breaks the hardware rule.

A real-time endpoint cannot be converted into a serverless endpoint. A serverless endpoint can be converted into a real-time endpoint, but the change cannot be undone. Cold starts depend on model size, download time, and

container startup. The OverheadLatency metric shows the launch time. Provisioned concurrency buys speed for predictable bursts.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/serverless-endpoints.html and https://docs.aws.amazon.com/sagemaker/latest/dg/deploy-model-options.html

Asynchronous inference

Fact: An asynchronous endpoint queues requests. The caller puts the payload in S3 and sends a pointer. SageMaker AI writes the result to an S3 output location and can send success or error notices through SNS. Payloads can reach 1 GB. Processing can run up to one hour.

Fact: An endpoint configuration with an AsyncInferenceConfig object accepts only asynchronous calls. The course slides also describe a custom queue built with SNS or SQS in front of a real-time endpoint. That design is a separate pattern.

Rule: Choose asynchronous when payloads are large, processing is slow, and the caller can wait for an S3 result.

Trap: Asynchronous is not a live chat API. A user who waits for an immediate reply needs real-time or serverless hosting.

Trap: Asynchronous does not process a full dataset as one offline job. A nightly load of a whole archive points to batch transform.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/async-inference.html and https://docs.aws.amazon.com/sagemaker/latest/dg/deploy-model-options.html and Sundog course slide 319

Batch transform

Fact: Batch transform runs inference over a dataset stored in S3. It does not need a persistent endpoint. Processing can take days, and datasets can reach gigabytes. Each input file produces one output file with an .out suffix, in the same record order. Use it for preprocessing too.

Fact: Set SplitType to Line to create mini-batches. MaxPayloadInMB must not exceed 100 MB. The product of MaxConcurrentTransforms and

MaxPayloadInMB must also stay within 100 MB. One input file goes to one instance, so extra instances sit idle when only one file exists.

Rule: Choose batch transform when a full dataset runs once or on a schedule and no endpoint must stay up.

Trap: Batch transform does not answer a single user request. Each live request needs an endpoint of some kind.

Batch transform also helps when you test model variants. Create one job per variant with its own output path. Compare the output files with the same validation set. The docs say batch transform does not support CSV input that contains embedded newline characters.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/batch-transform.html and https://docs.aws.amazon.com/sagemaker/latest/dg/deploy-model-options.html

Multi-model endpoints

Fact: A multi-model endpoint hosts several models behind one endpoint. The course slides show models A, B, and C sharing one endpoint. Each endpoint keeps its own deployment guardrails, VPC settings, and network isolation.

Fact: Model Monitor does not support multi-model endpoints. Serverless endpoints do not support them either.

Rule: Choose a multi-model endpoint when many similar, small models share traffic patterns and cost matters more than per-model scaling.

Trap: A multi-model endpoint cannot give each model its own scaling policy. Choose inference components for that need.

Source: Sundog course slide 317 and https://docs.aws.amazon.com/sagemaker/latest/dg/model-monitor.html and https://docs.aws.amazon.com/sagemaker/latest/dg/serverless-endpoints.html

Inference components

Fact: An inference component is a hosting unit for one model on a shared endpoint. The course slides say each model gets its own scaling policy.

Compute settings and copy counts per component are Not in source for this module.

Fact: The real-time endpoint documentation lists fine-tuning with adapter inference components as a topic. The maximum number of components per endpoint is Not in source. The fetch for the component page returned only a title, so this module does not quote its settings.

Rule: Choose inference components when several models of different sizes must share one real-time endpoint and scale separately.

Trap: A multi-model endpoint is a different design. It shares one endpoint across models without per-model scaling policies.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/realtime-endpoints.html and Sundog course slide 317

Lifecycle, approval, and governance

SageMaker Model Registry

Fact: The Model Registry catalogs models in model package groups. Each training result registers as a new version in the group. You attach metadata, lineage, and approval status to each version. Collections group several model groups. CI/CD pipelines can deploy approved versions only.

Fact: The fetched page confirms approval status and deployment automation, but it does not list the status values. The exact status names are Not in source for this module. Check the CreateModelPackage reference before you quote them.

Rule: Use Model Registry when a reviewer must approve a version before any deployment and an audit trail is required.

Trap: S3 versioning keeps old files. It does not hold an approval state that a pipeline can check before deploy.

Trap: The registry records what was approved. It does not watch the live endpoint for drift. Model Monitor does that job.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/model-registry.html and Sundog course slides 331, 332

SageMaker Pipelines

Fact: SageMaker Pipelines defines a directed acyclic graph of ML steps. You can build it in the visual pipeline designer or write JSON. It runs inside SageMaker AI and integrates with SageMaker Unified Studio. The list of available step types is Not in source for this module.

Rule: Use Pipelines when the same ML steps must run again with each new dataset or model version.

Trap: Pipelines orchestrates ML steps. A workflow that mixes human waits with many business services points to Step Functions instead.

Pipelines and the Model Registry work together. A pipeline registers each candidate as a version. A later step deploys only the version that a reviewer approved. Event names and rule patterns for that trigger are Not in source for this module.

Source: Sundog course slide 341 and https://docs.aws.amazon.com/sagemaker/latest/dg/model-registry.html

Lineage tracking and model cards

Fact: SageMaker ML Lineage Tracking records the workflow of data, jobs, and models as linked entities. It supports audit and compliance. Lineage can cross accounts through AWS Resource Access Manager. The LineageQuery API finds the models and endpoints that use a given artifact.

Fact: Model cards document intended use and evaluation for a model. The course slides mention a model card creation view for governance. Card fields and API limits are Not in source for this module.

Rule: Use lineage when an auditor must trace a deployed model back to its data and training job.

Trap: Lineage shows where a model came from. It does not record whether a reviewer approved it. The registry handles approval.

Source: Sundog course slides 333, 335, 433

Monitoring and fairness

SageMaker Model Monitor

Fact: Model Monitor watches a deployed model on a real-time endpoint or a batch transform job. It compares live data with a baseline built from training data. Monitor types are data quality, model quality, bias drift, and feature attribution drift. Results go to S3 and metrics go to CloudWatch.

Fact: Model Monitor works on tabular data only. It supports single-model endpoints only. Data capture stops at high disk use, so keep disk use below 75 percent. The docs say Model Monitor is no longer open to new customers, and AWS plans no new features.

Rule: Use Model Monitor for drift on a deployed single-model endpoint when the account already uses it and a baseline exists.

Trap: Model Monitor is closed to new customers. A design that starts a new monitoring program should name a different path, and this module does not list one. Not in source.

Feature attribution drift uses a rank score that compares the feature order in training data with the order in live data. Model quality drift can use ground truth labels to compare predictions with outcomes. Bias drift repeats the Clarify bias metrics on live data and sends alerts.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/model-monitor.html and Sundog course slides 326, 329, 330

SageMaker Clarify

Fact: Clarify measures bias in a dataset before training. The pre-training metrics include class imbalance, difference in proportions of labels, several divergence

measures, and conditional demographic disparity. Clarify also explains which features drive a prediction.

Rule: Use Clarify when the question is about group fairness or which features drive a prediction.

Trap: Clarify does not filter unsafe text or block a response. That job belongs to a guardrail service, which is covered in another module.

Clarify and Model Monitor connect for bias drift. Clarify computes the bias metrics. Model Monitor runs them on a schedule against live traffic. Alerts arrive through CloudWatch, and the team reviews the report before it retrains.

Source: Sundog course slides 327, 328, 433

Bedrock or SageMaker AI

Pick Bedrock when the need is a managed foundation model reached through an API. Pick SageMaker AI when the need is control of the container, the instance, the endpoint type, or the training loop. The scenario usually says who owns the weights. That sentence often decides the service.

Bedrock Custom Model Import sits between the two. It takes weights you own and serves them through Bedrock APIs. The course slides describe this path: train or tune in SageMaker AI, then import into Bedrock. A scenario that only needs invocation with your weights does not need a SageMaker endpoint.

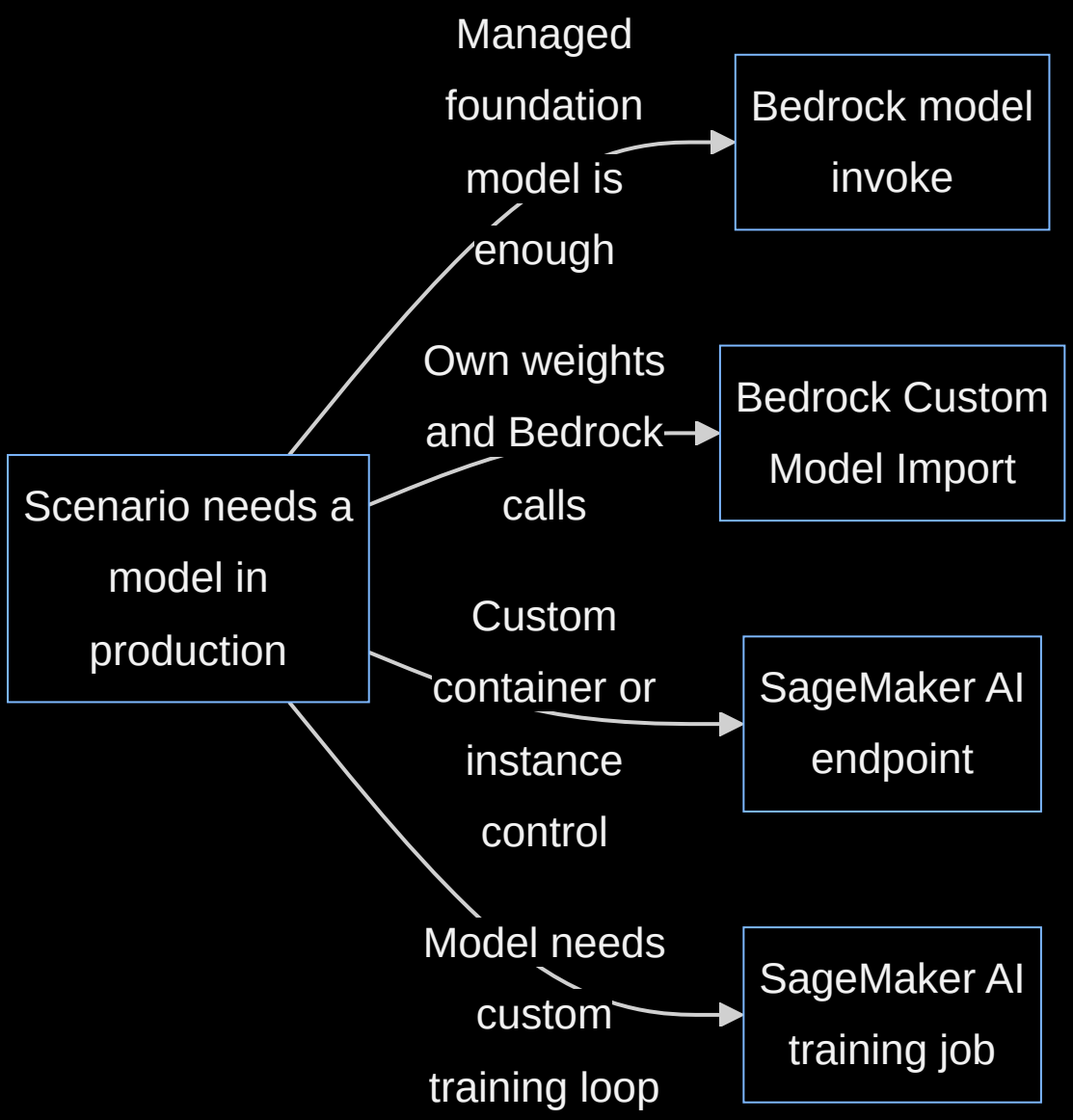


Figure: Ownership of weights and the need for hosting control decide the service.

Choosing SageMaker AI over Bedrock

- Rule: Choose Bedrock when a hosted foundation model and its API meet the requirement.
- Rule: Choose SageMaker AI when you need a custom container, a custom instance type, or a non-standard endpoint.
- Rule: Choose SageMaker AI when a model needs a custom training loop or a large-scale cluster.
- Rule: Choose Custom Model Import when you own the weights and want the Bedrock invocation surface.

Exam tip: The phrases "custom container", "endpoint type", "batch", and "pipeline" usually point to SageMaker AI. The phrases "managed", "API only",

and "no infrastructure" usually point to Bedrock.

Trap: A scenario that says "deploy on Bedrock" does not mean the team needs a SageMaker endpoint. It also does not mean the team can deploy any container. The model must fit the supported list.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/model-customization-import-model.html and Sundog course slide 317

Provisioned Throughput and custom model invocation

Fact: Provisioned Throughput reserves model units for a model at a fixed hourly cost. It is billed by the hour until you delete it. You can commit for no term, one month, or six months. A longer commitment lowers the hourly price.

Fact: If you customized a Bedrock model, you must buy Provisioned Throughput before you can use it. The pricing of a custom model matches the base model it came from.

Rule: For a Bedrock fine-tuned model, plan for Provisioned Throughput and not for on-demand calls.

Trap: A Custom Model Import model uses on-demand invocation through InvokeModel. Do not carry the fine-tuned model rule over to imported models without checking.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html and https://docs.aws.amazon.com/bedrock/latest/userguide/model-customization-import-model.html

---

Versus pairs: confusable choices

Each pair below has a decision tree, a three-row comparison, one question to ask, and the trap that catches most readers. Read the question first. Then decide which row of the comparison the scenario needs.

Fine-tuning versus continued pre-training

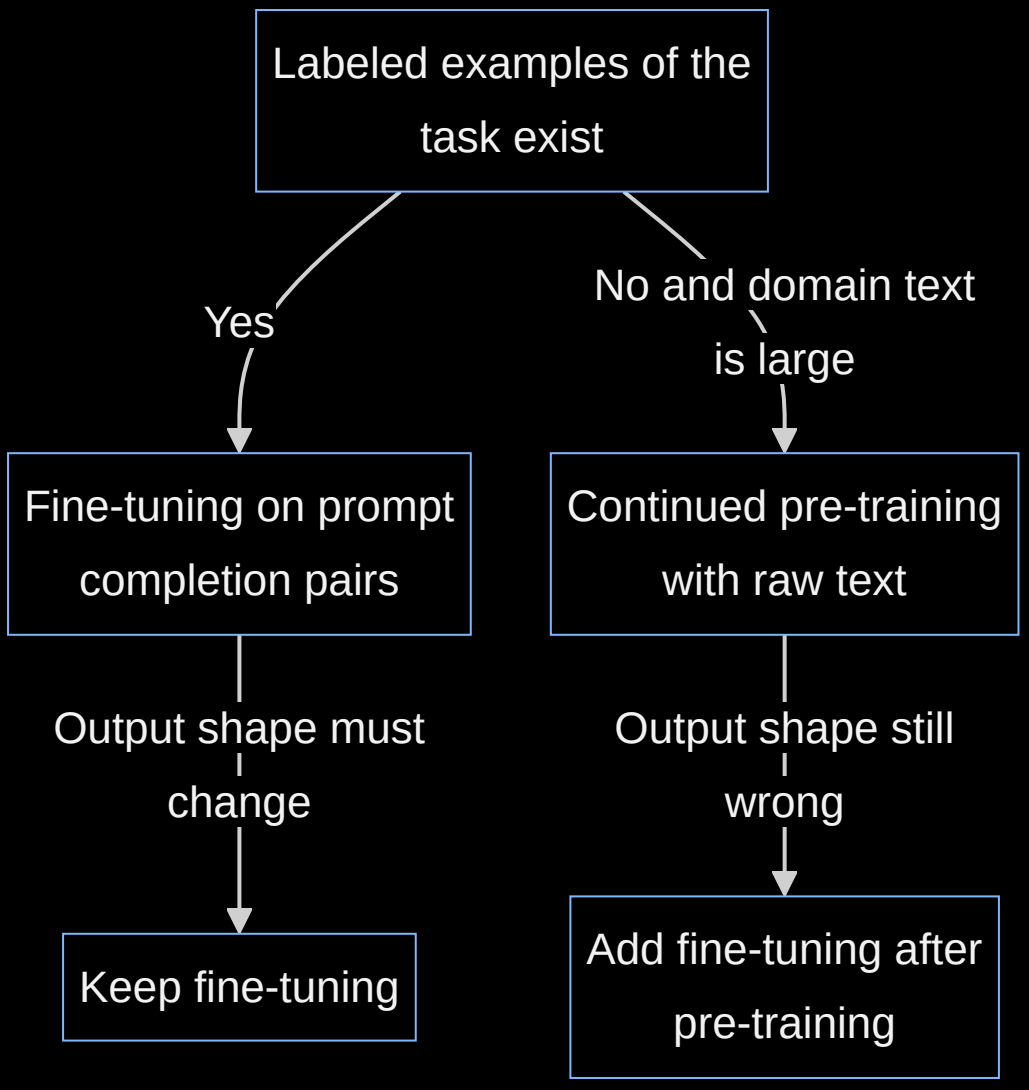


Figure: Labels decide the method. Raw text without labels points to pre-training.

- **Purpose:** Fine-tuning teaches a task. Continued pre-training teaches domain language.
- **Input:** Fine-tuning uses labeled records. Continued pre-training uses unlabeled text records.
- **Effect:** Fine-tuning changes task output. Continued pre-training widens general domain knowledge.

Question to ask yourself: Does the data have a correct answer for each record?

Trap: Continued pre-training is not a cheaper fine-tuning. It needs more text and does not fix a required format by itself.

Retrieval versus fine-tuning

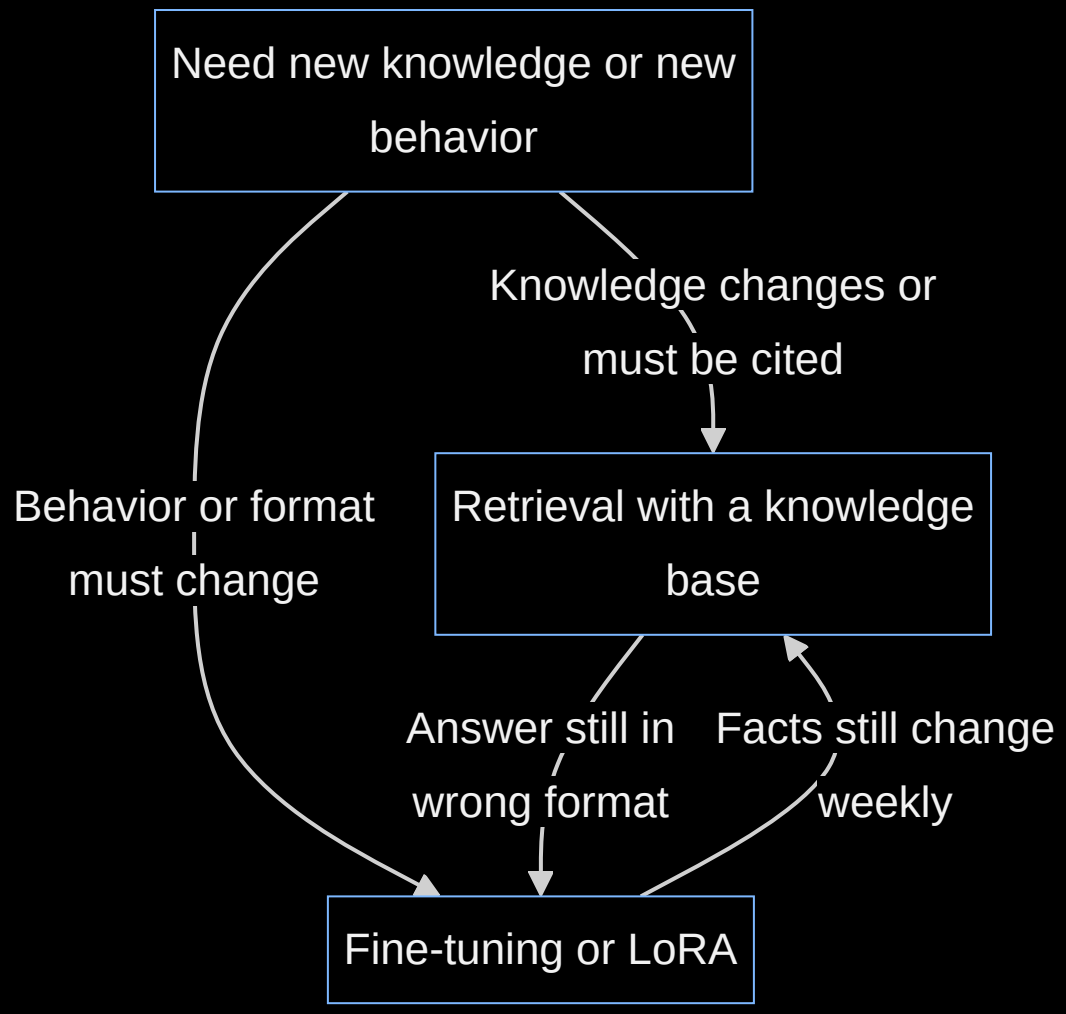


Figure: Facts and behavior are different problems. Many designs need both.

- **Purpose:** Retrieval supplies current facts at request time. Fine-tuning changes how the model responds.
- **Input:** Retrieval reads a document index. Fine-tuning reads labeled examples.
- **Effect:** Retrieval updates when the index updates. Fine-tuning changes only after a new training run.

Question to ask yourself: Does the model need a new fact, or a new habit?

Trap: Fine-tuning a model on policy documents looks like a fix. The facts then go stale inside the weights, and each update needs a new training run.

Distillation versus fine-tuning

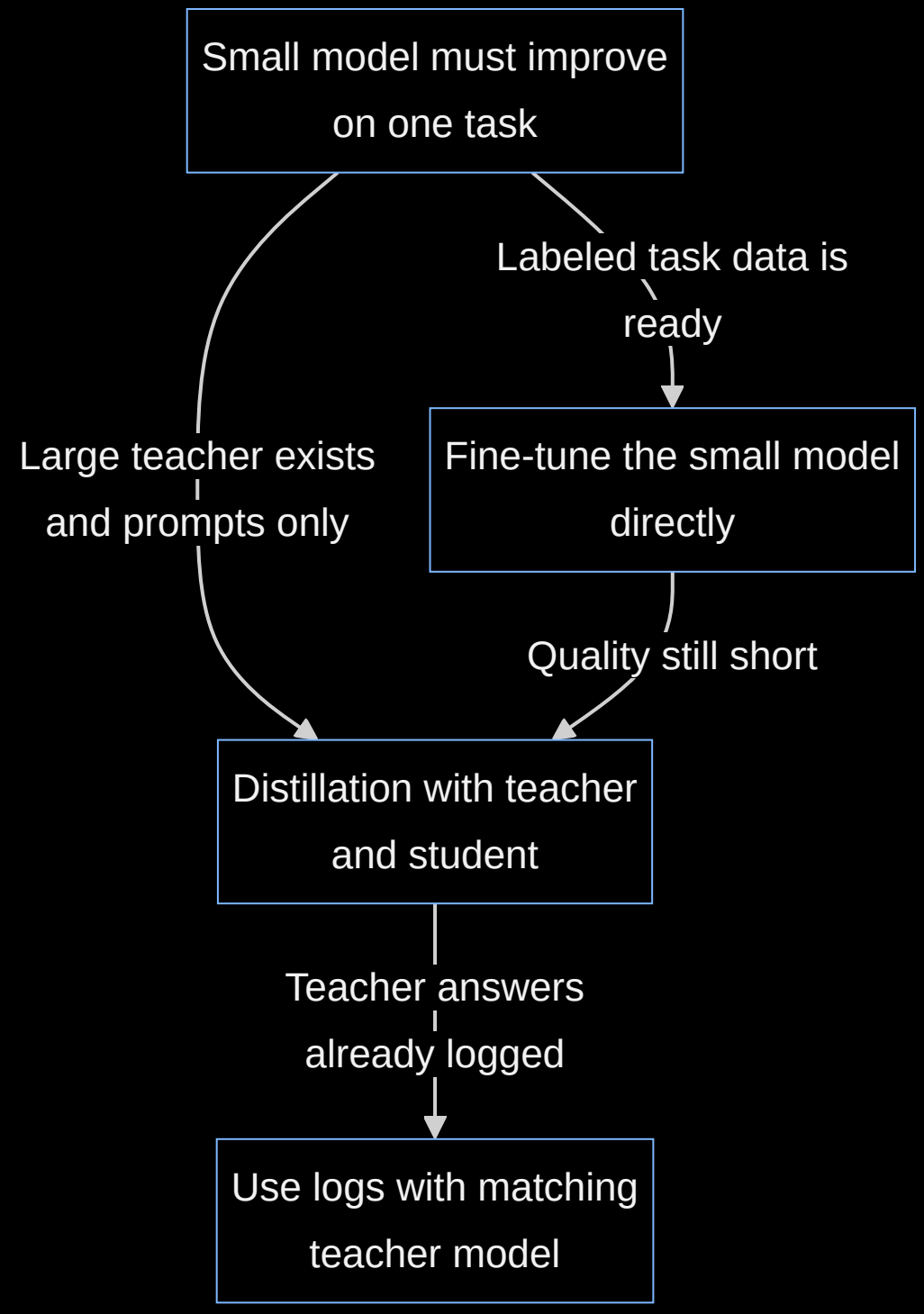


Figure: A teacher model is the difference. Distillation copies behavior from a larger model.

- **Purpose:** Distillation copies the behavior of a large model into a small model. Fine-tuning teaches a small model from labels.
- **Input:** Distillation starts from prompts, pairs, or logs. Fine-tuning starts from labeled pairs.
- **Effect:** Distillation lowers cost per call on the task. Fine-tuning changes the task skill of the small model.

Question to ask yourself: Is there a bigger model whose answers are the target?

Trap: Distillation is not a way to make a model smarter than its teacher. The student copies the teacher on the task.

Bedrock customization versus SageMaker AI training

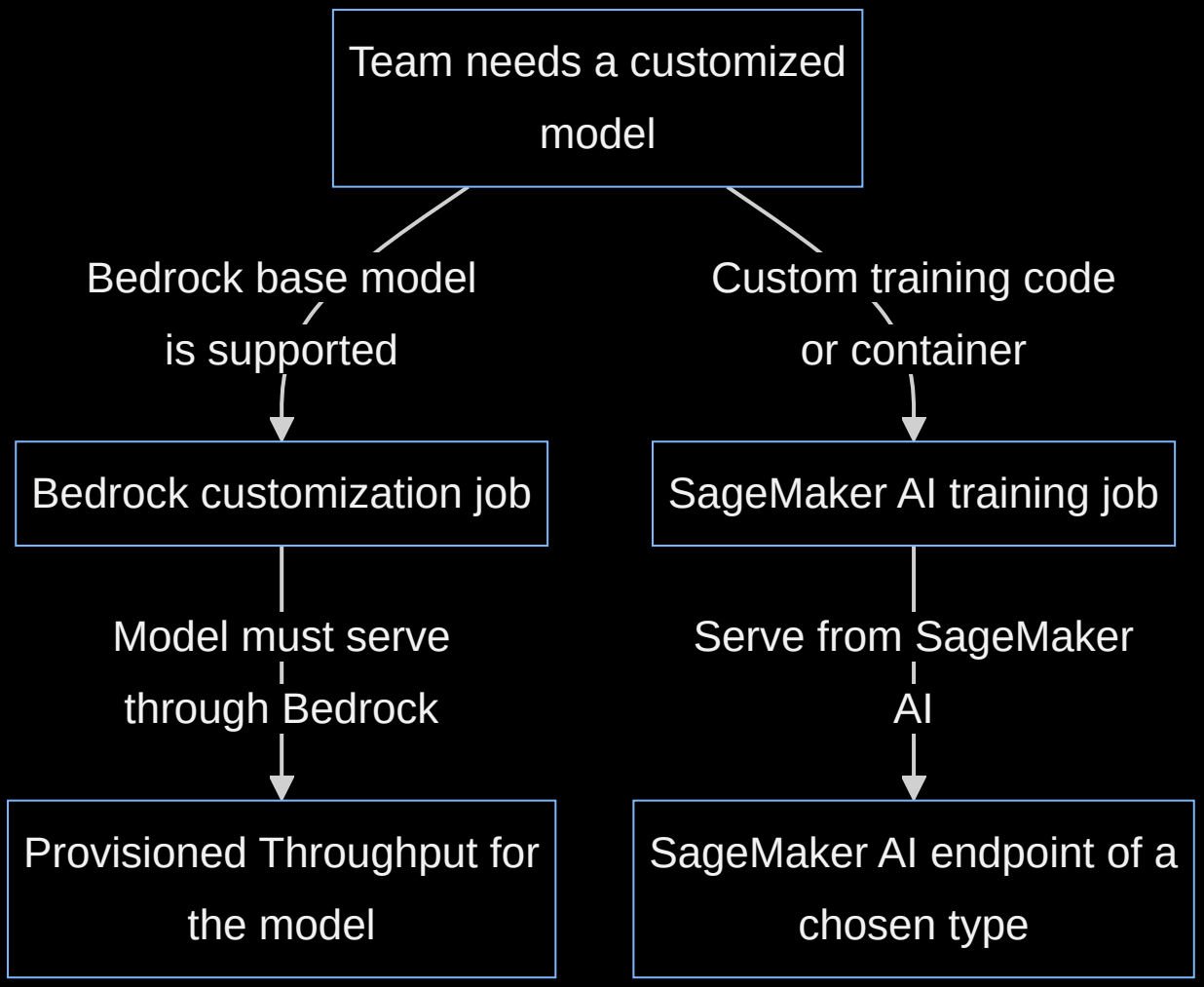


Figure: The base model support list and the need for custom code decide the path.

- **Purpose:** Bedrock customization tunes a supported base model in a managed job. SageMaker AI training runs your own code.
- **Input:** Bedrock uses JSONL in S3 that matches the method. SageMaker AI uses S3 data and your container.
- **Effect:** Bedrock returns a custom model for Bedrock invocation. SageMaker AI returns an artifact for an endpoint you choose.

Question to ask yourself: Does the scenario need code or a container that Bedrock cannot run?

Trap: A SageMaker AI training job alone does not serve traffic. The team must still choose and pay for an endpoint.

Custom Model Import versus Bedrock fine-tuning

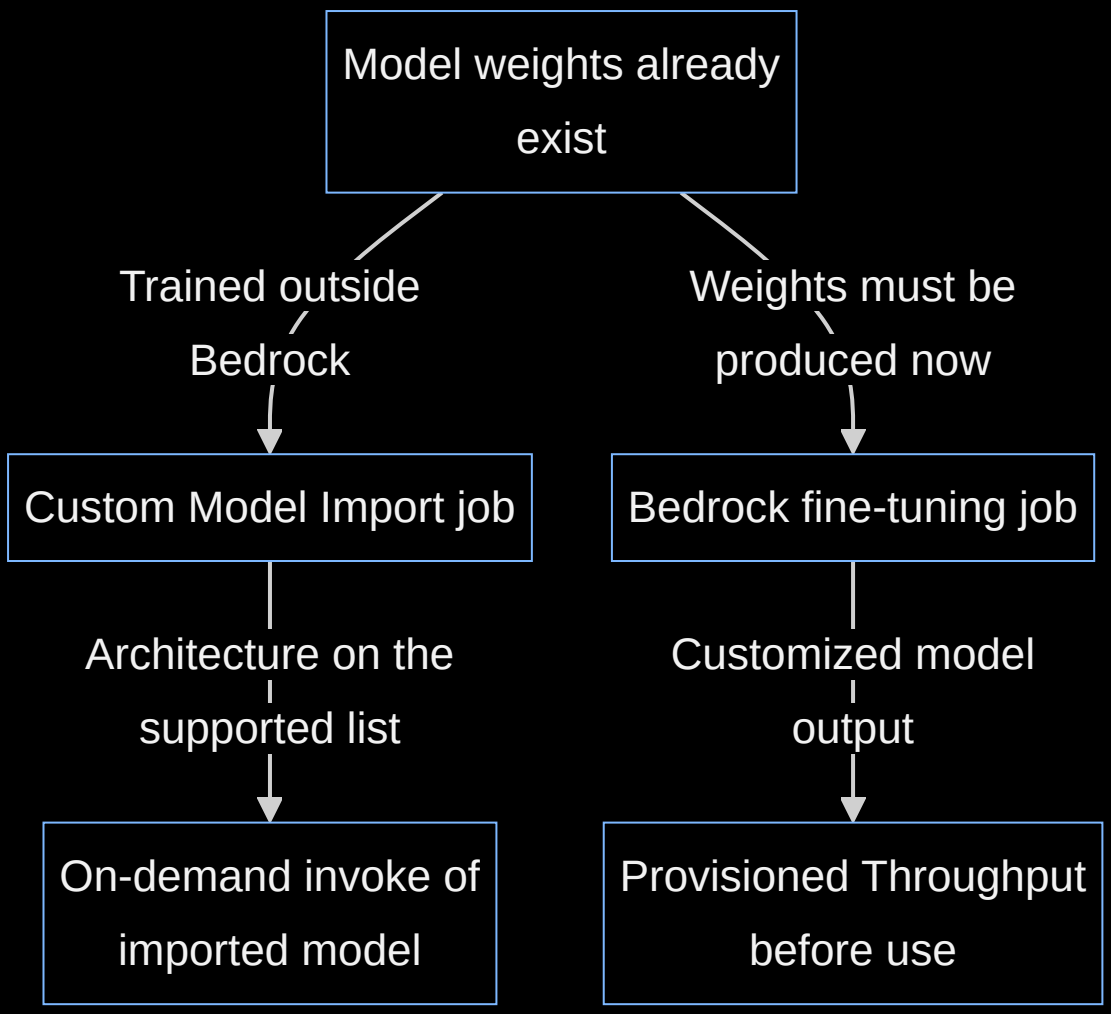


Figure: Ownership of trained weights comes first. Import hosts them. Fine-tuning creates them.

- **Purpose:** Import hosts weights you already trained. Fine-tuning trains new weights in Bedrock.
- **Input:** Import takes Hugging Face format weights in S3. Fine-tuning takes labeled JSONL in S3.
- **Effect:** Import supports on-demand calls for supported architectures. Fine-tuning needs Provisioned Throughput for use.

Question to ask yourself: Does the team already have the trained weights?

Trap: Import cannot run a training job. A scenario that asks for new training needs a customization job or SageMaker AI training.

Real-time endpoint versus asynchronous endpoint

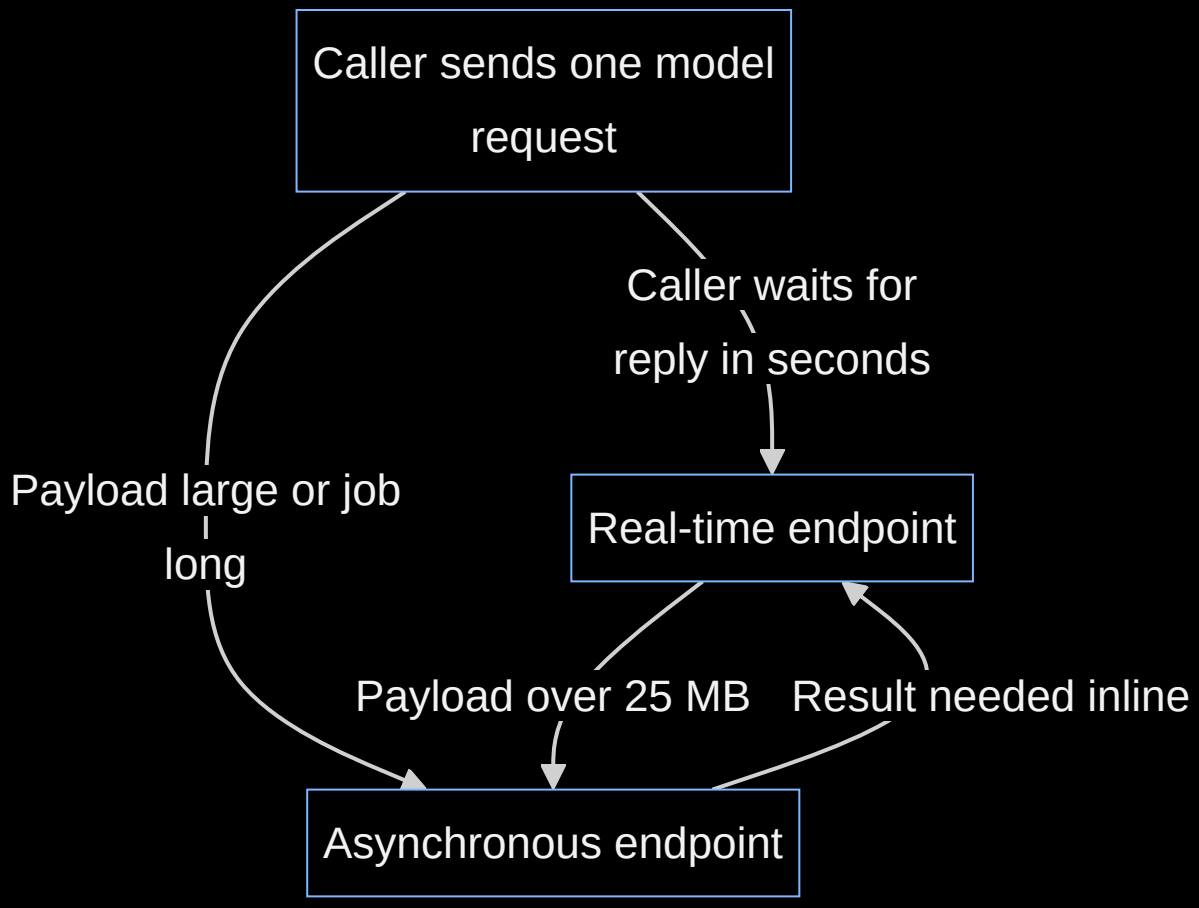


Figure: Payload size and wait tolerance decide the two endpoint types.

- **Purpose:** Real-time answers each call inline. Asynchronous queues calls and writes results to S3.
- **Input:** Real-time takes a payload up to 25 MB. Asynchronous takes a payload up to 1 GB from S3.
- **Effect:** Real-time must finish within 60 seconds for regular responses. Asynchronous can run up to one hour and scale to zero.

Question to ask yourself: Can the caller wait for an S3 result instead of a reply on the same connection?

Trap: Asynchronous is not a faster real-time endpoint. It is a queue with a longer wait.

Serverless versus real-time endpoint

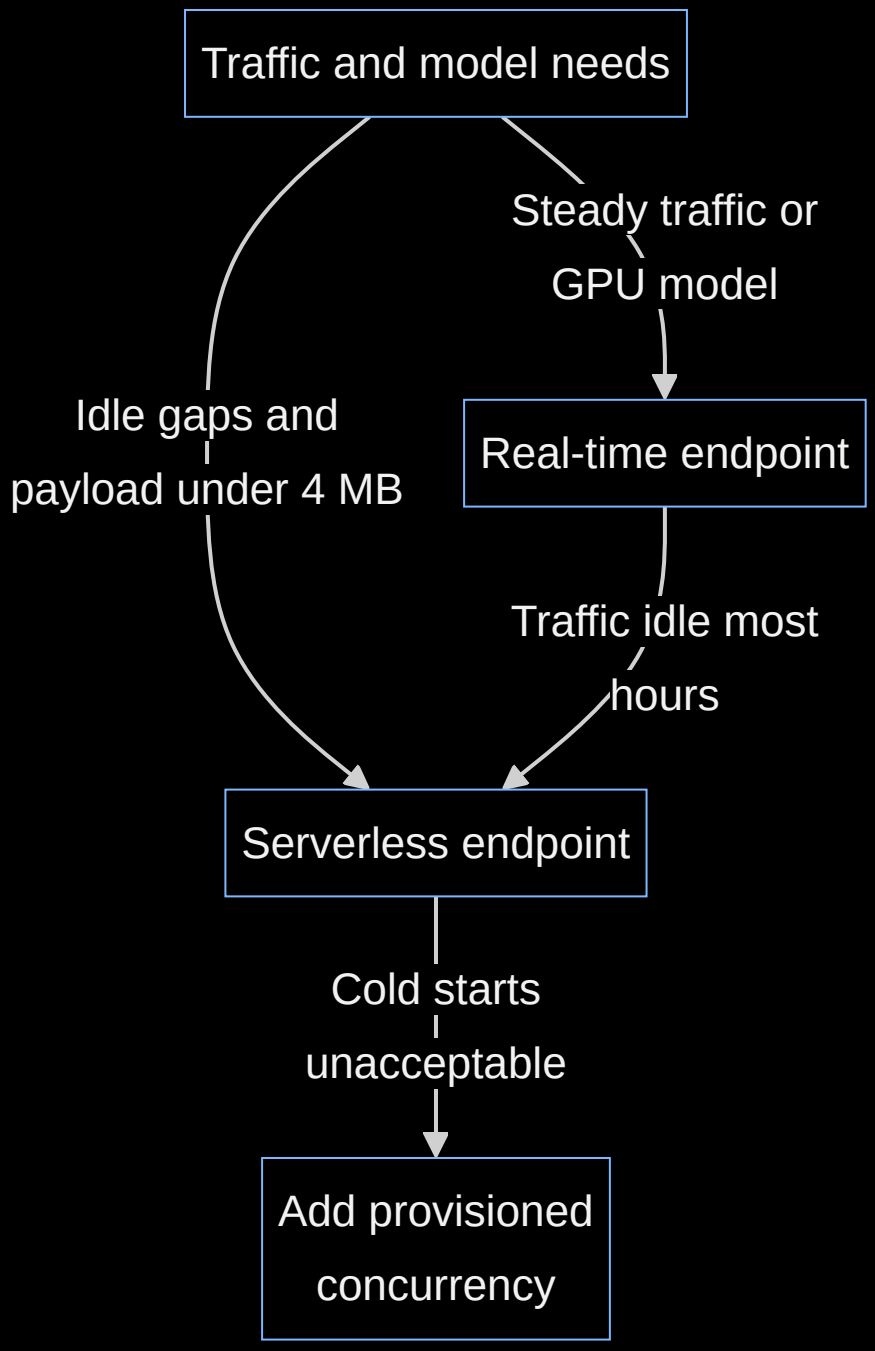


Figure: Idle time, payload size, and GPU need decide between serverless and real-time.

- **Purpose:** Serverless scales to zero with no instance choice. Real-time keeps a chosen instance running.
- **Input:** Serverless takes payloads up to 4 MB. Real-time takes payloads up to 25 MB.
- **Effect:** Serverless has no GPU and cold starts. Real-time has GPU options and steady latency.

Question to ask yourself: Does the model need a GPU or a payload over 4 MB?

Trap: Serverless does not mean unlimited. Its 60-second processing limit and memory cap still apply.

Batch transform versus asynchronous inference

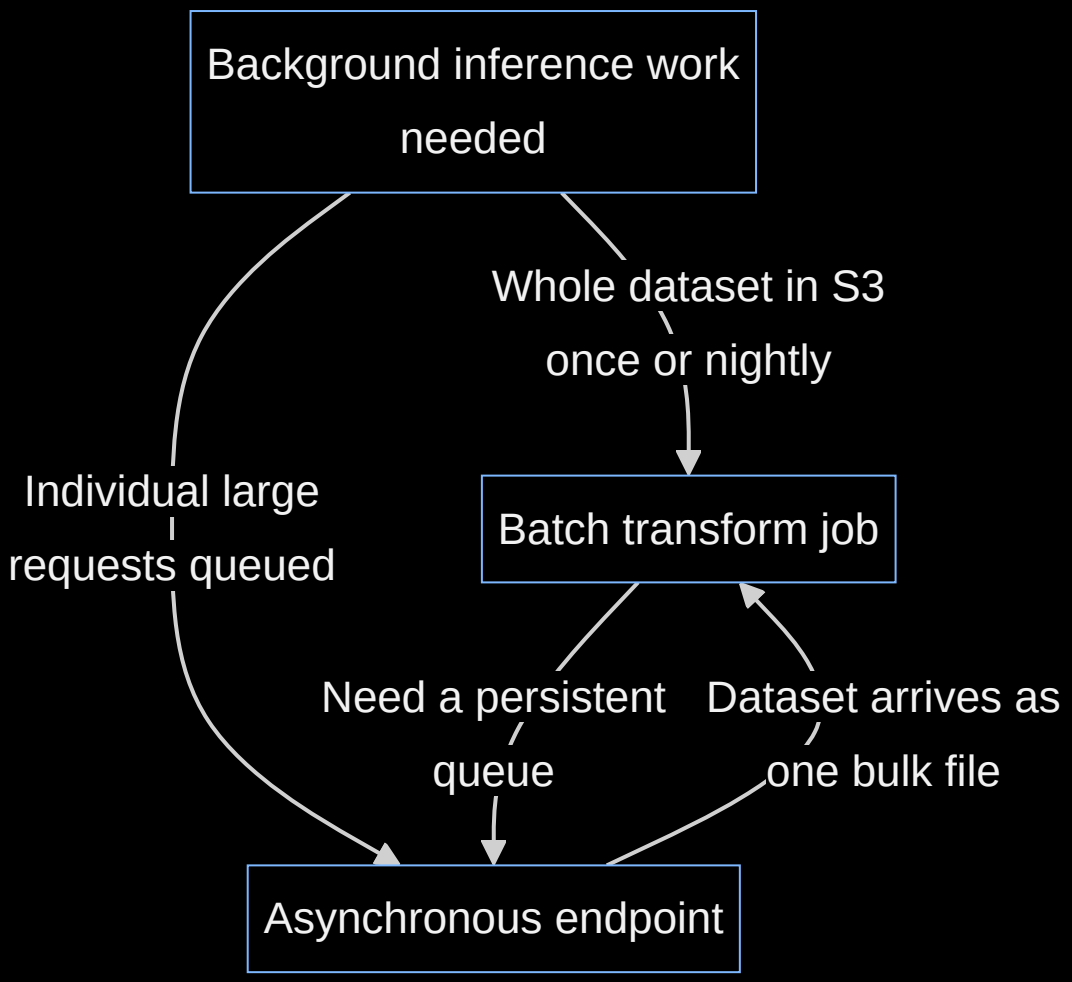


Figure: Batch transform runs a dataset. Asynchronous runs individual requests from a queue.

- **Purpose:** Batch transform runs a whole dataset and stops. Asynchronous serves requests as they arrive.
- **Input:** Batch transform reads S3 files split by line. Asynchronous reads one payload per request from S3.
- **Effect:** Batch transform writes one output file per input file. Asynchronous writes one output per request.

Question to ask yourself: Do requests arrive one by one, or is a whole dataset ready now?

Trap: Batch transform keeps no endpoint. A design that needs an endpoint for later calls cannot use it.

Multi-model endpoint versus inference components

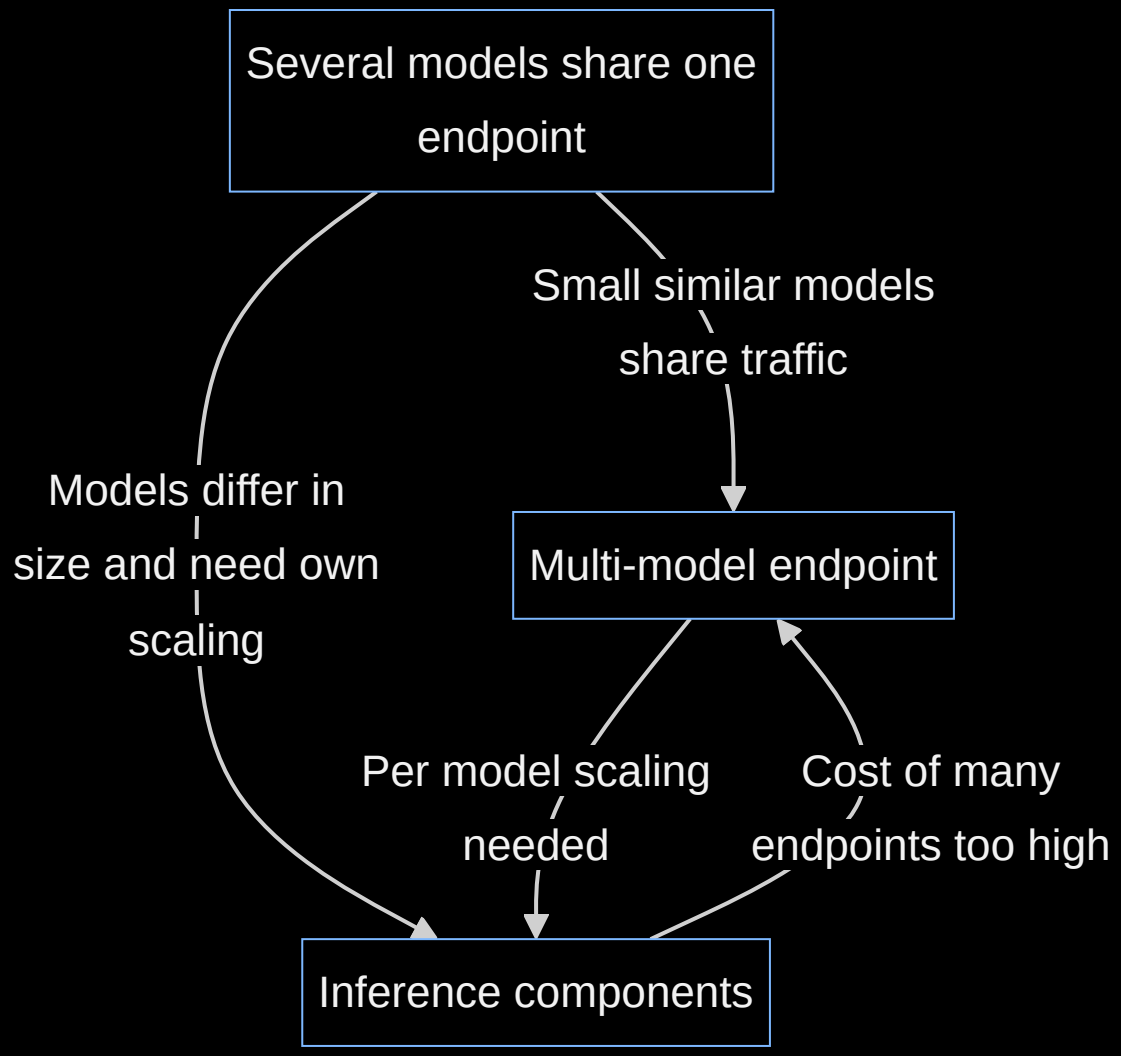


Figure: Per-model scaling is the deciding feature between the two designs.

- **Purpose:** A multi-model endpoint shares one endpoint across models. Inference components give each model a scaling unit on a shared endpoint.
- **Input:** Multi-model endpoints serve several model artifacts. Components serve model specifications with compute settings.
- **Effect:** Multi-model endpoints save cost for small models. Components give separate scaling for large models.

Question to ask yourself: Does each model need its own scaling policy?

Trap: A multi-model endpoint looks like the answer to every shared-endpoint question. It fails when models need separate scaling.

Model Monitor versus Clarify

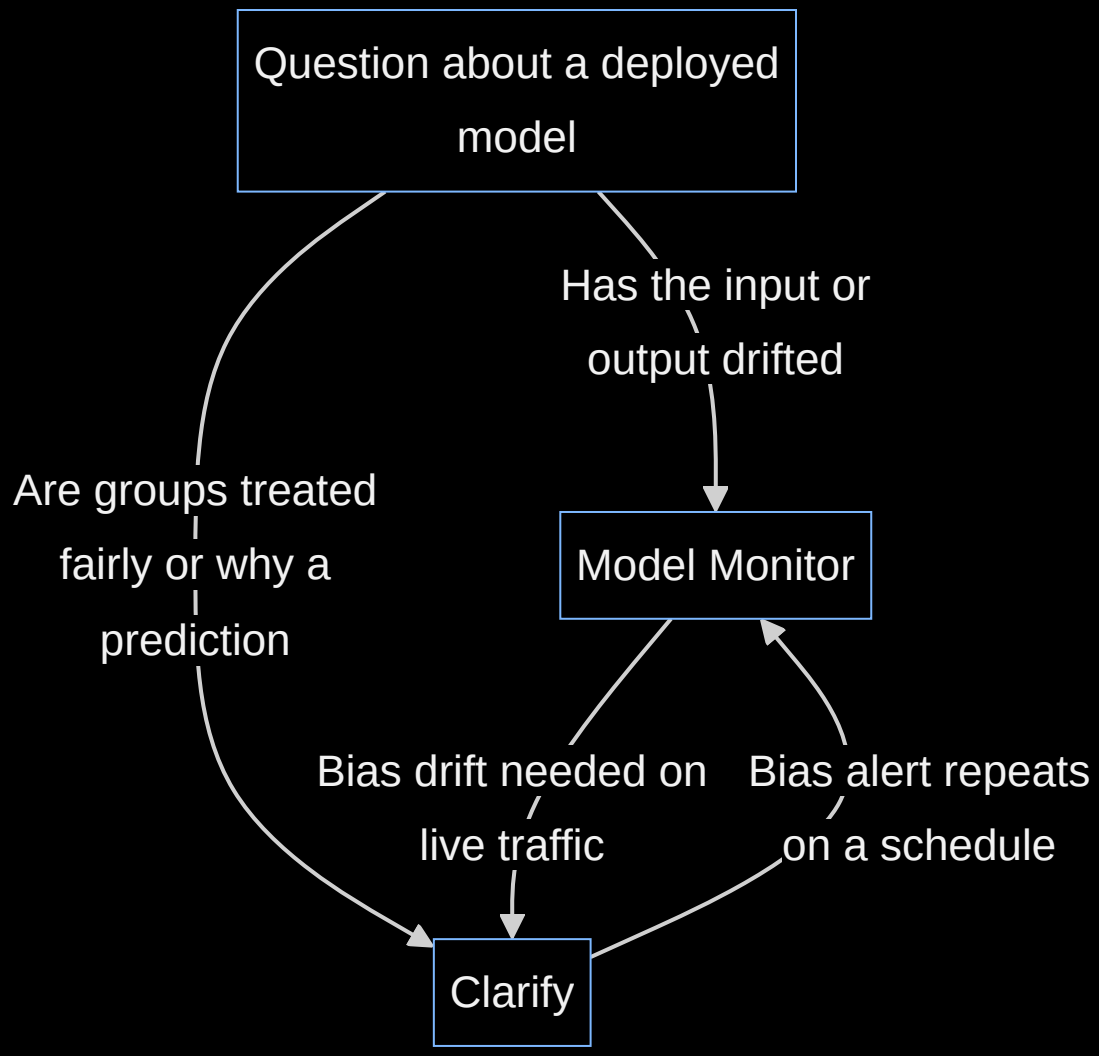


Figure: Drift over time points to Model Monitor. Fairness and explanation point to Clarify.

- **Purpose:** Model Monitor detects drift on a live model. Clarify measures bias and explains features.
- **Input:** Model Monitor reads captured requests and a baseline. Clarify reads a dataset or a model.
- **Effect:** Model Monitor sends drift metrics to CloudWatch. Clarify returns bias metrics and feature attributions.

Question to ask yourself: Is the question about change over time, or about fairness at a point in time?

Trap: Clarify alone does not watch a live endpoint on a schedule. Model Monitor does that with Clarify metrics.

Model Registry versus S3 versioning

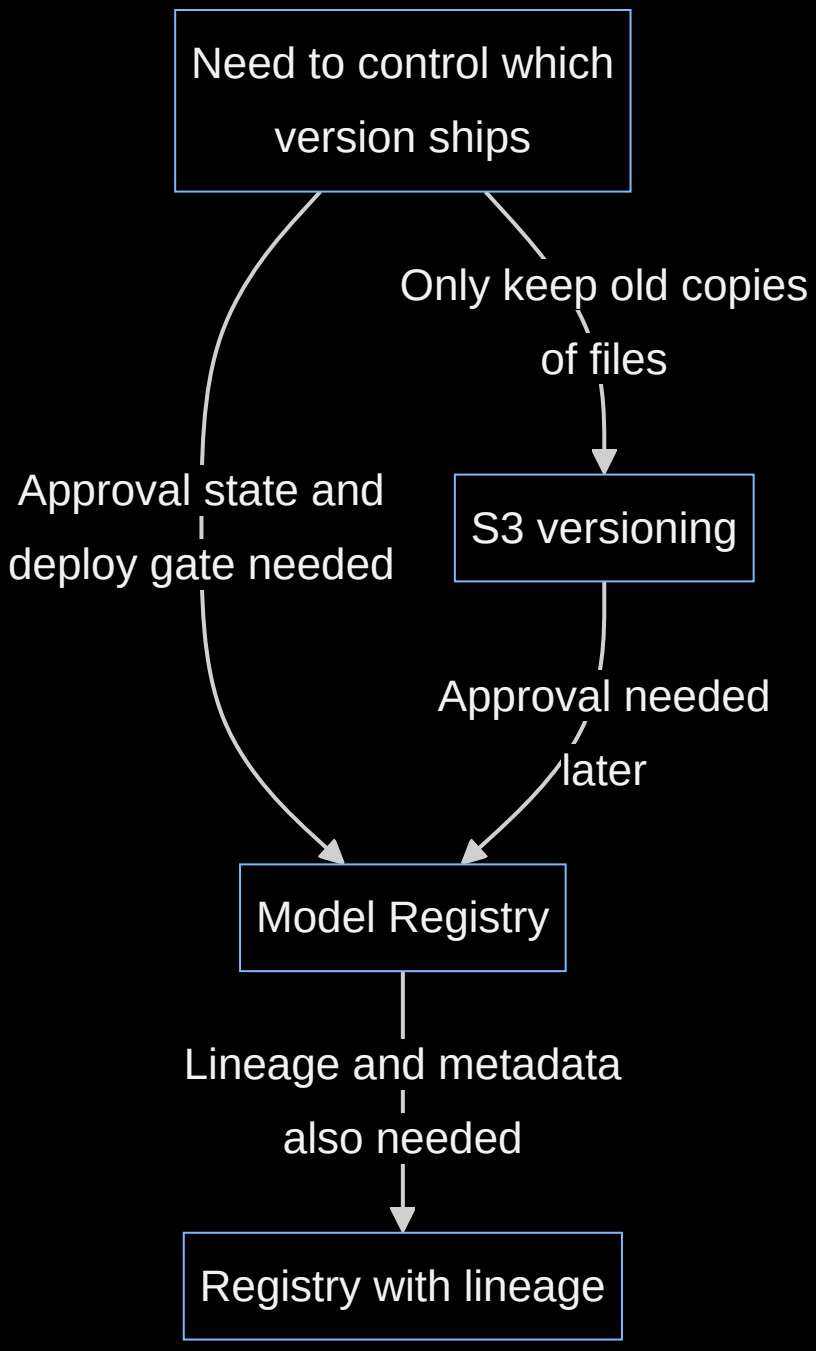


Figure: Storage keeps files. The registry keeps versions with approval and metadata.

- **Purpose:** The registry tracks model versions and approval. S3 versioning keeps every object revision.
- **Input:** The registry takes model packages with metadata. S3 takes any object.
- **Effect:** The registry lets a pipeline deploy only approved versions. S3 lets you restore an older file.

Question to ask yourself: Does a person or process need to approve a version before it ships?

Trap: S3 versioning plus a naming rule is not an approval workflow. It cannot block deployment of an unapproved file.

Read a long question

Each worked scenario below uses short, original sentences. Underline the constraint words first. Then remove each wrong option by type, with one reason. The answer is the option that meets the deciding constraint and nothing else in the list breaks.

Worked scenario one: long audio analysis

A team hosts its own speech model. Each request is a 120 MB recording, and the caller can wait about twenty minutes for a transcript stored in S3. The team does not want to pay for instances during quiet hours.

Underline: 120 MB, wait twenty minutes, S3 result, quiet hours.

- Trap: Real-time endpoint fails on payload size. Its limit is 25 MB and its regular response limit is 60 seconds.
- Trap: Serverless endpoint fails on payload size and processing time. Its limits are 4 MB and 60 seconds.
- Trap: Batch transform fails the request pattern. It is offline and does not serve individual callers.
- Trap: A bigger instance on a real-time endpoint fails the time limit. Instance size does not change the 60-second rule.

Answer: An asynchronous endpoint. It takes payloads up to 1 GB, runs up to one hour, writes to S3, and scales to zero.

Worked scenario two: smaller model for a single task

A company calls a large hosted model for ticket routing. The routing answers are correct, but the cost per call is high. The team has a library of prompts and no labeled answers, and it wants a smaller model for the same routing task.

Underline: smaller model, same task, prompts only, no labels, cost per call.

- Trap: Continued pre-training fails the task. It uses unlabeled text to widen domain knowledge, not to copy one task.
- Trap: RAG fails the cost goal. It adds context to each call and does not shrink the model.
- Trap: Fine-tuning on labels fails the no-labels fact. The team does not have labeled answers.
- Trap: Quantization shrinks the number format. It does not transfer the routing behavior of the large model.

Answer: Distillation with the large hosted model as teacher and the smaller model as student. Prompts are enough as input.

Worked scenario three: release control for tuned versions

A team tunes adapters on one base model every two weeks. A reviewer must approve each version before production. Auditors need to trace a live version back to its training data.

Underline: adapters, reviewer approves, before production, trace back to training data.

- Trap: S3 versioning fails the approval gate. It keeps files but does not block a deploy on approval state.
- Trap: Model Monitor fails the release control. It watches drift on a live model after release.
- Trap: A deployment tool such as CodeDeploy fails the model lifecycle. It moves application code and does not track model versions.
- Trap: A fresh endpoint per version fails the audit goal. It hides which version was approved and why.

Answer: Register each adapter release in Model Registry with approval status and lineage. Deploy only approved versions through a pipeline. Keep the base model as its own versioned entry.

Distractor patterns seen in the tests

Wrong options follow patterns. Learn the pattern and you can reject an option even when you do not know the service detail. Each pattern below names the error and gives the reason it fails. Use the patterns as a checklist after you find the constraint.

- **Bigger hammer:** More compute is offered for a limit that compute cannot change. A payload size or a fixed timeout stays the same on a larger instance.
- **Right idea, wrong layer:** Retrieval is offered to change tone, or tuning is offered to add changing facts. Each tool changes a different thing.
- **Storage as lifecycle:** Files kept in S3 are offered as version control. Storage keeps objects, but it does not gate a deployment on approval.
- **Monitoring as governance:** A drift monitor is offered as the approval record. Monitoring reports change after release, not who approved a version.
- **Wrong label type:** Labeled records are offered for raw text training, or raw text is offered for task training. Each method needs its own data shape.
- **Wrong API surface:** A control plane endpoint is offered for runtime calls. Inference traffic uses the runtime endpoint, not the management endpoint.
- **Serverless means limitless:** A serverless endpoint is offered for a GPU model or a long job. It keeps 4 MB and 60-second limits and has no GPU.
- **Speed by changing the model:** Quantization is offered when the question asks for the same quality. Quantization can change output quality, so it fails a quality constraint.
- **Client shape mismatch:** An application is told to send a new request shape to a hosted model. SageMaker AI does not reshape JSON for the client, so the app must format it.
- **Public path left open:** An option grants permission but leaves traffic on a public route. A private design needs interface endpoints and a route that stays inside the VPC.

- **Scale to zero on the wrong type:** A real-time endpoint is offered as scale to zero. Scale to zero belongs to serverless and asynchronous types.
- **Import mistaken for training:** A model import is offered as a way to tune weights. Import only hosts weights that already exist.

Drill

Read each prompt and answer before you look at the next line. Each answer names the deciding constraint and the one service it points to.

1. A policy library changes monthly, and every answer must cite its source document. Which approach fits?

Rule: Retrieval over the document index, because facts change and citations matter. Tuning does not fix changing facts.

2. A model must always return a fixed JSON layout, and the team has labeled examples. Which method fits?

Rule: Fine-tuning with labeled prompt and completion records, because the output shape must change.

3. A company holds 40 GB of unlabeled manuals and has no task labels. Which method fits?

Rule: Continued pre-training on the unlabeled text, because the goal is domain language, not one task.

4. A small model must copy the answers of a large model on one classifier task, and the team has prompts only. Which method fits?

Rule: Distillation with the large model as teacher, because prompts alone can drive teacher answers.

5. Requests carry 200 MB files and take fifteen minutes, and the caller can wait for an S3 result. Which endpoint fits?

Rule: An asynchronous endpoint, because payloads reach 1 GB and processing reaches one hour.

6. Traffic is idle most of the day, spikes at lunch, and payloads are 1 MB. Which endpoint fits?

Rule: A serverless endpoint, because it scales to zero and accepts payloads up to 4 MB.

7. A nightly job must score two terabytes of stored records once. Which option fits?

Rule: Batch transform with S3 input and output, because no persistent endpoint is needed for one run.

8. A team fine-tuned a Bedrock model and wants to serve it on demand without buying capacity. Is that possible?

Trap: No. A customized model needs Provisioned Throughput before use, so on-demand serving does not fit.

9. Weights trained outside AWS use a supported architecture, and the app must call Bedrock. Which path fits?

Rule: Custom Model Import, then invoke the imported model on demand with InvokeModel.

10. A reviewer must approve each model version before any production deployment, and an audit trail is required. Which feature fits?

Rule: Model Registry approval status, with a pipeline that deploys only approved packages.

11. A live single-model endpoint must alert when input features drift from training data. Which feature fits, and what must you check?

Rule: Model Monitor data quality with a baseline. Trap to check: the account must still be able to use it.

12. A team must measure whether training outcomes differ across demographic groups before any training starts. Which feature fits?

Rule: Clarify pre-training bias metrics, because the question is fairness in the dataset.

13. A company stores many tuned variants of one large base model and wants cheap storage and fast rollback. Which technique fits?

Rule: LoRA adapters registered as versions over one base model, because each adapter is small.

14. A real-time endpoint must serve 30 MB images. Is that within the limit?

Trap: No. The real-time payload limit is 25 MB. Use an asynchronous design or reduce the payload.

Sources

Source: AWS documentation fetched for this module:

- https://docs.aws.amazon.com/bedrock/latest/userguide/custom-models.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/model-customization-prepare.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/model-customization-import-model.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/model-distillation.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html
- https://docs.aws.amazon.com/sagemaker/latest/dg/deploy-model-options.html
- https://docs.aws.amazon.com/sagemaker/latest/dg/realtime-endpoints.html

- https://docs.aws.amazon.com/sagemaker/latest/dg/serverless-endpoints.html
- https://docs.aws.amazon.com/sagemaker/latest/dg/async-inference.html
- https://docs.aws.amazon.com/sagemaker/latest/dg/batch-transform.html
- https://docs.aws.amazon.com/sagemaker/latest/dg/model-registry.html
- https://docs.aws.amazon.com/sagemaker/latest/dg/model-monitor.html
- https://docs.aws.amazon.com/sagemaker/latest/dg/sagemaker-hyperpod.html
- https://docs.aws.amazon.com/sagemaker/latest/dg/studio-jumpstart.html

Source: Course slide extract used as a second source: m6 Sundog slide extract (slides 25 to 28, 57, 78, 92, 139, 300 to 301, 308, 311 to 319, 326 to 335, 341, 433).

All modules Previous Next